Real-Time Schedulers for STM32: Performance Evaluation of EDF over SuperLoop

Vladimir Toporkov, Artem Burmyakov
Innopolis University

I. Introduction

STM32 is a family of microcontrollers developed by STMicroelectronics for embedded applications. These devices integrate a processor, memory, and peripheral interfaces on a single chip [1].

STM32-based devices support applications such as environmental monitoring [2] and equipment condition monitoring [3]. In these applications maintaining responsiveness as the workload increases is a key challenge.

The paper examines how the choice of scheduling approach affects responsiveness and processing cost on STM32. We compare two approaches: SuperLoop [4] and Earliest Deadline First (EDF) [5].

A. Contribution

We compare periodic SuperLoop and chunked EDF on an STM32F767. We evaluate the effects of workload utilization and chunk duration on response time, deadline misses, skipped releases, and measured selection overhead. Our implementations for both schedulers are publicly available.

II. Background on STM32

Fig. 1 illustrates the sensor platform considered in this work, based on an STM32F767 uniprocessor. Table I summarizes the functions and interfaces of its four sensors.

TABLE I
Sensor functions and interfaces.

Sensor	Purpose	Interface
DHT11 [6]	Temperature and humidity	Single-wire digital (GPIO)
HC-SR04 [7]	Distance	Trigger/echo (GPIO)
OV2640 [8]	Image acquisition	Parallel data, SCCB
SPW2430 [9]	Signal acquisition	Analog output (ADC)

General-purpose input/output (GPIO) pins carry digital signals, while an analog-to-digital converter (ADC) converts an analog signal into digital values. The Serial Camera Control Bus (SCCB) configures the camera.

A. Programming principles

1.AB: To describe the key aspects of STM32 programming. Limit the text to one page.

1) Programming environment: software name, protocols, etc.
2) Template structure of the project (macroses) in the development environment;
3) Functionality (macroses) to be implemented in that structure;
4) Put some print screens.

III. System Model

We rely on a classical real-time periodic system model by Liu and Layland [5]. We adopt the task and release notation of Burmyakov and Bini [10].

A real-time system $\mathcal{T} = \{\tau_1, \ldots, \tau_n\}$ consists of n periodic tasks executed on a uniprocessor. Each task $\tau_i = (C_i, D_i, P_i)$ has a worst-case execution time C_i, a relative deadline D_i, and a period P_i. We consider implicit deadlines, $D_i = P_i$. Task utilization is $U_i = {}^{C_i}/_{P_i}$, and task-set utilization is $U_{\mathcal{T}} = \sum_{\tau_i \in \mathcal{T}} U_i$.

Let $J_{i,j}$ denote the j-th job of τ_i, released at $r_{i,j}$. Unlike the sporadic model in the referenced paper, consecutive releases are separated by exactly P_i:

$$r_{i,j+1} = r_{i,j} + P_i. \tag{1}$$

Job $J_{i,j}$ requires at most C_i units of CPU time and has an absolute deadline $r_{i,j} + D_i$. Let $f_{i,j}$ denote its completion time, if it completes. Release indices include jobs skipped by the implementation. These jobs have no completion time.

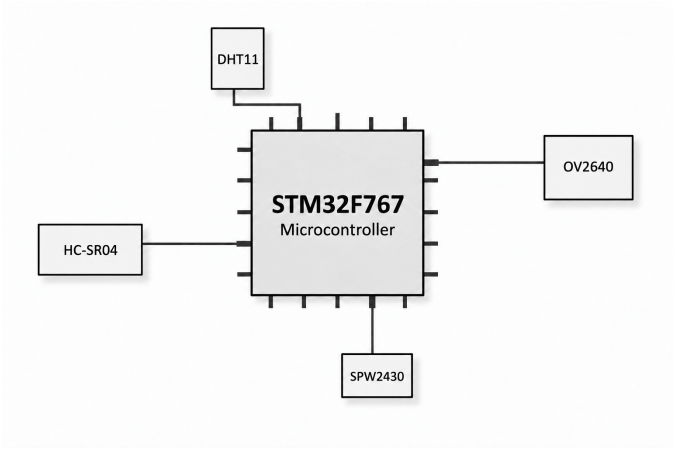


Fig. 1. Sensor platform based on STM32F767.

TABLE II
SYSTEM MODEL NOTATION.

Symbol	Meaning
$\mathcal{T}, n$	Task set and number of tasks
τ_i	Task i
C_i	Worst-case execution time of a job of τ_i
P_i	Task period
$D_i = P_i$	Relative deadline, equal to the period
$J_{i,j}$	The j-th job of task τ_i
$r_{i,j}$	Release time of job $J_{i,j}$
$f_{i,j}$	Completion time of an executed job
$r_{i,j} + D_i$	Absolute deadline of job $J_{i,j}$
$U_i = C_i/P_i$	Task utilization
$U_\mathcal{T} = \sum_{\tau_i \in \mathcal{T}} U_i$	Task-set utilization

- A schedule specifies which job executes on the processor at each time instant, or whether the processor executes no job.
- Preemption interrupts a job before completion to execute another job. The interrupted job may resume later.
- A completed job $J_{i,j}$ has missed its deadline if $f_{i,j} > r_{i,j} + D_i$.
- The response time of a completed job $J_{i,j}$ is $f_{i,j} - r_{i,j}$, the time from its release to completion.

Table II summarizes the notation introduced above. The evaluation plots use U as shorthand for $U_\mathcal{T}$.

IV. SCHEDULERS

We consider two schedulers: SuperLoop and chunked EDF.

A. SuperLoop

The key steps of the SuperLoop scheduler are outlined in Algorithm 1.

Algorithm 1 SuperLoop Scheduler

```
function SUPERLOOP(T)
    t_i^next ← 0,  i = 1, ..., n                ▷ Initial releases
    while true do
        for i ← 1 to n do                        ▷ Fixed task order
            t^cur ← NOW
            if t_i^next ≤ t^cur then
                J_{i,j} ← RELEASEJOB(τ_i, t_i^next)
                EXECUTE(J_{i,j})                 ▷ Non-preemptive
                f_{i,j} ← NOW                    ▷ Completion time
                t_i^next ← t_i^next + P_i
                while t_i^next < f_{i,j} do      ▷ Equality retained
                    t_i^next ← t_i^next + P_i
                end while
            end if
        end for
    end while
end function
```

Algorithm 2 Chunked EDF Scheduler

```
function CHUNKEDEDF(T, Δ)
    t_i^next ← 0,  i = 1, ..., n                ▷ Initial releases
    while true do
        t^cur ← NOW
        for all τ_i ∈ T do                       ▷ Check releases
            if t_i^next ≤ t^cur and τ_i has no active job then
                RELEASEJOB(τ_i, t_i^next)        ▷ Add to R
            end if
        end for
        if R ≠ ∅ then                            ▷ Pending work
            J_{i,j} ← SELECTEDF(R)               ▷ Earliest deadline
            EXECUTECHUNK(J_{i,j}, Δ)             ▷ At most Δ
            if COMPLETED(J_{i,j}) then
                f_{i,j} ← NOW
                COMPLETEJOB(J_{i,j}, f_{i,j})    ▷ Skip rule
            end if
        end if
    end while
end function
```

B. Earliest Deadline First

V. PERFORMANCE EVALUATION

A. Experimental setup

Experiments run on the NUCLEO-F767ZI at 96 MHz. Physical sensors are disabled in these experiments. Each synthetic job executes a timed busy-wait workload. The scheduler also uses busy polling. Timing is measured using the processor cycle counter in the Data Watchpoint and Trace (DWT) unit. Reports are transmitted over a serial interface (UART) after measurement stops.

The results below use three tasks with $(P_1, P_2, P_3) = (10, 50, 200)$ ms and $D_i = P_i$. Table III lists the configured synthetic workload durations, which are nominal budgets rather than measured worst-case bounds. Thus, $U_\mathcal{T}$ denotes nominal workload utilization and excludes instrumentation and scheduling overhead. Both schedulers use the same task set. SuperLoop polls tasks in index order. The illustrative schedules assume synchronous first releases at $t = 0$, whereas the implementation uses $r_{i,1} = t_0 + P_i$, one period after initialization.

TABLE III
THREE-TASK SYNTHETIC WORKLOADS. EXECUTION BUDGETS ARE IN μS.

Nominal $U_\mathcal{T}$ (%)	C_1	C_2	C_3
50	1667	11111	22222
65	2167	14444	28889
75	2500	16667	33332
80	2667	17778	35556
90	3000	20000	40000
95	3167	21111	42222
100	3333	22222	44444

The main comparison uses 60 s windows and $\Delta = 1$ ms for EDF. The chunk-size study uses $\Delta \in \{1, 2, 4\}$ ms at

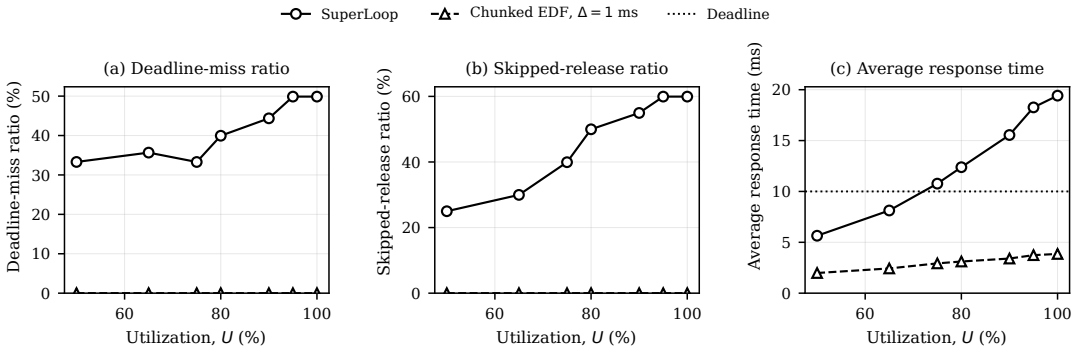


Fig. 2. Performance for τ_1: three synthetic tasks, $\Delta = 1$ ms for EDF, 60 s windows.

$U_{\mathcal{T}} \in \{50, 90, 100\}\%$. The figure below shows $U_{\mathcal{T}} = 90\%$. Each configuration is run three times from reset. Additional campaigns use two tasks and 100 s windows to examine task-set and window-length effects. Extended diagnostic statistics are disabled in all these campaigns.

B. Evaluation metrics

Let N_i, M_i, and S_i be the numbers of completed jobs, completed jobs that missed their deadlines, and skipped releases of τ_i in a run. Let W be the requested measurement-window duration. We report:

- Deadline-miss ratio: $100M_i/N_i$ percent, defined for $N_i > 0$.
- Skipped-release ratio: $100S_i/(W/P_i)$ percent, normalized by the nominal number of releases in the requested window.
- Average response time: the mean of $f_{i,j} - r_{i,j}$ over completed jobs. The maximum observed response time is also recorded.
- Selection overhead: measured time spent selecting jobs in iterations that execute work, divided by the actual measurement duration, in percent. This excludes uninstrumented scheduler work and is not total processor utilization.

Measurement stops at the first loop boundary after W, so its actual duration may be slightly longer. Released but unfinished tracked jobs and their overdue status are recorded separately and excluded from completion-based metrics. Subsequent releases awaiting skip accounting are not included in this pending-job count. Plot points are arithmetic means of the corresponding per-run metrics over three runs, not pooled averages over all jobs.

C. SuperLoop and EDF comparison

Fig. 2 shows the results for τ_1. Under SuperLoop, its deadline-miss and skipped-release ratios generally increase with utilization, as does its average response time. In contrast, no deadline misses or skipped releases are observed under chunked EDF ($\Delta = 1$ ms) for τ_1 in the evaluated three-task configurations. EDF gives τ_1 precedence over jobs with later absolute deadlines, whereas SuperLoop follows the fixed polling order.

Compared with SuperLoop, chunked EDF yields a higher average response time for τ_3 in the evaluated configurations (Fig. 3). Its response time increases with utilization. Under EDF, jobs with earlier absolute deadlines execute first. At nominal $U_{\mathcal{T}} = 100\%$, no capacity is reserved for scheduling and execution overhead, and deadline misses and skipped releases are observed for τ_3 under chunked EDF. However, the lower response time under SuperLoop occurs alongside skipped releases of τ_1. It therefore does not imply better service for the entire task set.

D. Effect of chunk duration

Fig. 4 shows the effects of increasing Δ at $U_{\mathcal{T}} = 90\%$. Larger chunks reduce the frequency of scheduling decisions. The reported selection overhead decreases, whereas the average response time of τ_1 increases and that of τ_3 decreases. Longer non-preemptive chunks delay τ_1. The decrease for τ_3 may reflect changes in execution order and fewer scheduling interruptions. These overhead measurements remain preliminary pending validation of the instrumentation.

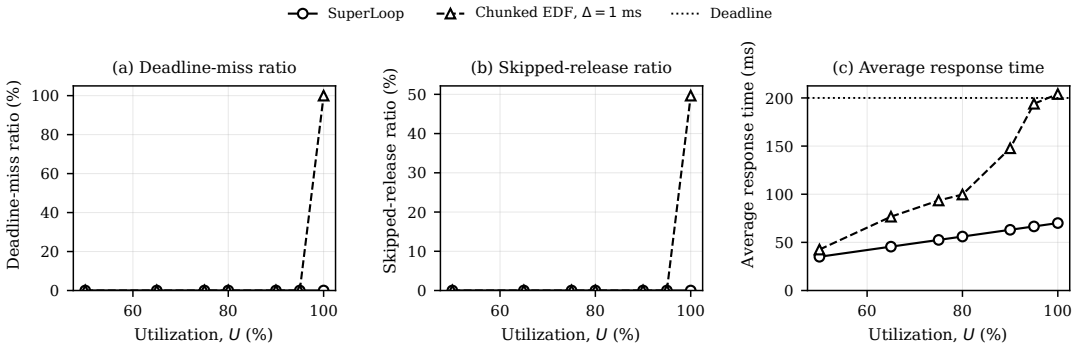


Fig. 3. Performance for τ_3 under the same conditions as Fig. 2.

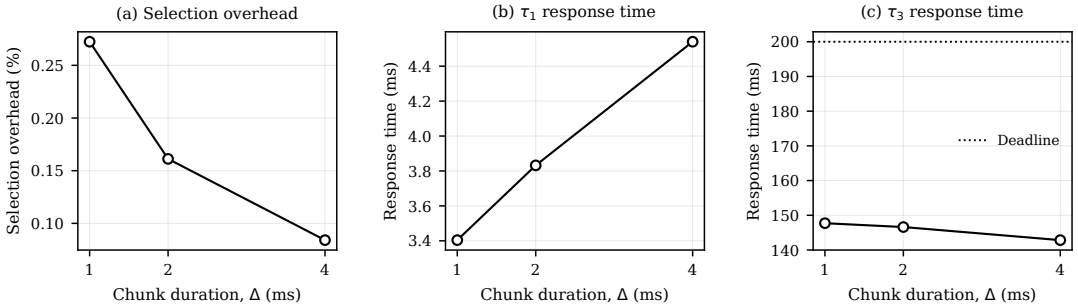


Fig. 4. Impact of chunk duration at $U_{\mathcal{T}} = 90\%$ with three synthetic tasks and 60 s windows.

REFERENCES

[1] *STM32F765xx, STM32F767xx, STM32F768Ax, STM32F769xx: Datasheet*, STMicroelectronics. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32f767zi.pdf

[2] STMicroelectronics, "Process and environment monitoring," accessed: Oct. 6, 2026. [Online]. Available: https://www.st.com/en/applications/factory-automation/process-and-environment-monitoring.html

[3] ——, "Condition monitoring / predictive maintenance," accessed: Oct. 6, 2026. [Online]. Available: https://www.st.com/en/applications/factory-automation/condition-monitoring-predictive-maintenance.html

[4] Arm Keil, "Execution models: Single task system (Superloop)," accessed: Oct. 4, 2026. [Online]. Available: https://www.keil.com/pack/doc/mw/Graphic/html/gui_exec_model.html

[5] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973.

[6] *DHT11 Humidity & Temperature Sensor: Technical Data Sheet*, Aosong Electronics. [Online]. Available: https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf

[7] *Ultrasonic Ranging Module HC-SR04*, Elecfreaks. [Online]. Available: https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf

[8] *OV2640 Datasheet*, OmniVision Technologies. [Online]. Available: https://www.uctronics.com/download/OV2640_DS.pdf

[9] *SPW2430HR5H-B: Top Port SiSonic Microphone*, Knowles Electronics. [Online]. Available: https://www.mouser.com/datasheet/2/218/SPW2430HR5H-B-1290924.pdf

[10] A. Burmyakov and E. Bini, "Improving the efficiency of global EDF exact schedulability test," *IEEE Transactions on Computers*, pp. 1–14, 2026.